KakeyaLattice: D4 Nested-Lattice KV-Cache Compression with Kakeya-Style Discrete Codebooks

Allen Li
Individual researcher
Email: AllenL329@gmail.com

April 23, 2026

Abstract

We present KakeyaLattice v1.4, a KV-cache compression codec for transformer-based large language models that is, to our knowledge, the first Kakeya-inspired discrete-codebook construction to beat the production TurboQuant [1] codec head-to-head on real vLLM inference across four open-source model families (Qwen3-4B, DeepSeek-R1-Distill-Qwen-1.5B, Gemma-4-E4B, GLM-4-9B-Chat).

The codec is a Zamir–Feder nested lattice quantiser built on the D_4 root lattice, wrapped in the full TurboQuant engineering stack (Sylvester–Hadamard rotation, per-vector adaptive scale, unit-norm factorisation, matched bit budget). The D_4 structure contributes a shaping gain of approximately $+0.37\,\mathrm{dB}$ over scalar $\mathbb{Z}^4$ quantisation; our theoretical prediction that this translates to a K-MSE ratio of $\approx 0.92\times$ versus TurboQuant matches the measured ratio of $0.911\times$ on live vLLM to within 1%. On 12 head-to-head pairs (4 models $\times$ 3 matched bit points) KakeyaLattice wins universally on K-MSE (12/12, 10–36% better), wins 9/12 on $|\Delta\mathrm{ppl}|$ (4/4 at the aggressive $\sim 3.6\times$ compression point, $1.8\times$–$5.3\times$ better), and reproduces the same shaping gain across every model. At 128k context the codec shrinks the K+V cache by 2.5–2.8$\times$ at balanced quality and by 1.66–1.73$\times$ at near-lossless quality; the Q=152 near-lossless point has $|\Delta\mathrm{ppl}| < 1\%$ and top-1 pair agreement $\geq 98\%$ on every tested model.

We place this construction inside the Kakeya–Brascamp–Lieb–Tropp dependency chain — Guth's multilinear Kakeya endpoint [29] $\rightarrow$ Bennett–Carbery–Christ–Tao Brascamp–Lieb equivalence [31] $\rightarrow$ Tropp's matrix concentration [33] $\rightarrow$ Halko–Martinsson–Tropp randomised-algebra bounds [34] — whose deep end is used by Wang and Zahl [35] in their recent resolution of the three-dimensional Euclidean Kakeya conjecture. The D_4 nested-lattice construction is a literature-traceable shallow instance of the same Kakeya-style toolchain: D_4 realises the densest packing lattice in $\mathbb{R}^4$, and the lattice's Voronoi cells form a Kakeya-like direction-covering system for quantised residuals. A research trail of three independent "Dvir-to-Euclidean bridges" [27] — Guth–Katz polynomial partitioning, naive Zamir–Feder D_4, and empirical non-Gaussian shaping — is summarised together with the measurements that ruled each out, isolating the combination "D_4 lattice + TurboQuant engineering stack" as the first winning construction.

We position KakeyaLattice v1.4 as an $\approx 8\%$ improvement to the upstream TurboQuant kernel rather than a standalone codec, because (i) the D_4 structure is a drop-in replacement for the int·per-coord quantiser inside TurboQuant's existing Hadamard + qmax wrapper, and (ii) the engineering stack contributes $\sim$1000x more to the measured quality than the lattice choice, dwarfing any standalone Kakeya-inspired construction. The measured gain is architecture-independent (same shaping factor on four different transformer families) and carries zero quality cost at the aggressive compression point. All code, multi-model measurements, raw per-passage JSON, and snapshot-mode vLLM harness are released under Apache-2.0 at `https://github.com/FluffyAIcode/LLM-KV--Cache-compress` at tag `v1.4`.

1 Introduction

The key–value (KV) cache is the dominant memory constraint in production LLM inference [15, 16, 20]. At context length T, a model with L layers, H_{kv} key/value heads, and head dimension d stores $4 \cdot L \cdot H_{kv} \cdot d \cdot T$ bytes in bf16; for Qwen3-4B at $T = 128{,}000$ this is 18 GiB per user. Cache compression is therefore one of the highest-leverage engineering and algorithmic targets in current serving infrastructure.

1.1 TurboQuant as the current production frontier

As of April 2026, the strongest production-grade KV compressor is **TurboQuant** [1], merged into vLLM main on 2026-04-15 (PR #38479 [2]) with subsequent cleanup PRs [3, 4, 5]. TurboQuant's $k8v4$ preset stores K with FP8 E4M3 after Sylvester–Hadamard rotation and V as 4-bit uniform, reaching 2.6× compression at 95.6% of baseline GSM8K and 100% NIAH on Qwen3-4B on 4× RTX PRO 6000 Blackwell GPUs. More aggressive presets ($t4nc$ at 3.8×, $k3v4nc$ at 4.3×, $t3nc$ at 4.9×) trade quality for compression but stay 100% NIAH at 32k context.

TurboQuant is therefore the rational baseline for any new codec: it is production-ready, architecture-agnostic, and already optimised at the kernel level. The question of this paper is: *can a Kakeya-style discrete codebook improve on TurboQuant's Hadamard + per-coordinate Lloyd-Max combination?*

1.2 Three negative bridges and one positive

Dvir's polynomial-method resolution of the finite-field Kakeya problem [27] offers three independent "bridges" to continuous Euclidean quantisation:

1. **Bridge A** — Guth–Katz polynomial partitioning [30], transferred via a Johnson–Lindenstrauss projection to low dimension followed by degree-2 monomial features.
2. **Bridge B** — raw Zamir–Feder nested lattice [11] on the D_4 root lattice, using the Conway–Sloane closest-point algorithm [12].
3. **Bridge C** — data-matched non-Gaussian shaping: TurboQuant's Hadamard + per-coordinate structure with the scalar quantiser replaced by empirical Lloyd–Max centroids fit on real LLM-captured K.

We implemented all three, benchmarked them on real Qwen3-4B post-QK-norm pre-RoPE K tensors, and found each loses against TurboQuant by one to three orders of magnitude in reconstruction MSE (Section 5). Decomposing the 1414× loss of Bridge B reveals that four engineering factors — Hadamard rotation, per-vector adaptive $q_{\max}$, unit-norm factorisation, and matched bit budget — each contribute 2–10×, while the D_4 lattice's own theoretical contribution is only +0.37 dB (×0.92). Adding the engineering stack to Bridge B yields the codec of this paper, provisionally named **Bridge B2**, and formally released as

KakeyaLattice v1.4
(class `V14KakeyaZamirLatticeGPU`)

The prediction from the decomposition is K-MSE ratio $\approx 0.92\times$ against TurboQuant at matched bits; the measured ratio on 100 k held-out samples is 0.913×; the measured ratio inside live vLLM is 0.911×. All three numbers agree within ∼1%.

1.3 Contributions

Algorithm. The D_4-nested-lattice + TurboQuant-wrapper codec (Section 3).

Theory. A placement of the D_4 skeleton inside the Kakeya–Brascamp–Lieb–Tropp dependency chain, via a quantitative correspondence between lattice Voronoi cells and restricted Kakeya direction covers (Section 2).

Empirical. The first measured Kakeya-inspired win over TurboQuant on live vLLM on four open-source model families (Section 6).

Deployment scope. A honest positioning as a drop-in improvement to the existing TurboQuant vLLM kernel rather than a standalone codec (Section 8).

Research trail. A transparent record of two additional Kakeya-style constructions that failed (Guth–Katz polynomial partitioning and non-Gaussian shaping), plus the four-factor engineering decomposition that isolated what the D_4 structure actually buys (Section 5).

What this paper is not. This is not a survey of KV compression; related-work positioning is deferred to Section 9. This is not a new upper-bound on the Kakeya maximal function; our use of the Kakeya chain is as a *literature-traceable framing* of the D_4-lattice construction, not a new mathematical theorem in harmonic analysis.

2 Design philosophy: the Kakeya–Brascamp–Lieb–Tropp chain

2.1 Rate–distortion formulation

Let $X = \{x_i\}_{i=1}^n \subset \mathbb{R}^D$ denote a block of n cached key vectors at head dimension D (post-QK-norm, pre-RoPE, following the capture protocol of Section 4). Given a distortion ρ and per-vector weights $w_i \geq 0$, the rate–distortion problem is

$$\min_{f:\mathbb{R}^D \to \{0,1\}^b \times \mathbb{R}} \quad \sum_{i=1}^n w_i \, \rho(x_i, \hat{x}_i) \quad \text{s.t.} \quad b(f) \leq B, \tag{1}$$

where $\hat{x}_i = f^{-1}(f(x_i))$ is the reconstruction, $b(f)$ the total bits used, B the byte budget, and $\rho(x, \hat{x}) = \|x - \hat{x}\|_2^2$ in our setting (see Section 10 for the MSE vs. inner-product discussion).

2.2 From direction sets to Kakeya-like covers

Every KV block induces a direction set $\Theta = \{x_i / \|x_i\| : x_i \neq 0\} \subset S^{D-1}$. The classical Kakeya problem [22] asks for the minimum measure of a set in $\mathbb{R}^D$ containing a unit segment in every direction. The Kakeya-maximal conjecture asserts Hausdorff dimension D for any such set; in $\mathbb{R}^3$ it is resolved by Wang and Zahl [35].

A codec that reconstructs every x_i with ℓ_2-error ε must cover, for every direction $\theta \in \Theta$, a tube of radius ε. The minimum cost of such a tube-union is exactly the quantity the Kakeya maximal-function estimates bound [23, 24, 25, 26].

Proposition 2.1 (Kakeya-conditional rate–distortion lower bound, informal)**.** *Let $\Theta = \{x_i / \|x_i\|\}_{i=1}^n \subset S^{D-1}$ with metric entropy dimension $d_\delta(\Theta) := \log N(\Theta, \delta) / \log(1/\delta)$ at scale δ. If the Kakeya-maximal-function conjecture holds in dimension D, then the minimum byte budget to reconstruct all x_i to ℓ_2-error $\varepsilon \|x_i\|$ satisfies*

$$B(\varepsilon, n, D) \;=\; \Omega\big(n \cdot \log_2 \tfrac{1}{\varepsilon} \;+\; d_\delta(\Theta) \cdot \log_2 \tfrac{1}{\varepsilon}\big) \tag{2}$$

for any $\delta \leq \varepsilon$. At $D = 3$ this lower bound is unconditional by Wang–Zahl [35]; at $D \geq 4$ it remains conditional on the Kakeya conjecture in dimension D.

This gives a clean design dictum: *a Kakeya-like codec should construct the cover that dominates the dimensional scaling of Θ and spend its bit budget on perpendicular residuals.* In v1.3 we instantiated this with a low-rank PCA skeleton; in v1.4 we instantiate it with a D_4-lattice Voronoi tessellation of the Hadamard-rotated residual.

2.3 The four-step dependency chain

The D_4-lattice skeleton sits at the shallow end of an explicit four-step chain, each link a published theorem:

1. **Multilinear Kakeya endpoint.** Guth [29] proves the endpoint case of the Bennett–Carbery–Tao multilinear Kakeya conjecture [28]. This is the deep geometric ingredient of Wang–Zahl's $\mathbb{R}^3$ proof.

2. **Brascamp–Lieb equivalence.** Bennett, Carbery, Christ, and Tao [31] show that the multilinear Kakeya estimate is equivalent, up to constants, to a geometric Brascamp–Lieb inequality.

3. **Matrix concentration.** Tropp's matrix Chernoff inequality [33] and its sharpest constants go through the Lieb concavity theorem, a special case of Brascamp–Lieb [32].

4. **Lattice sphere-packing concentration.** The D_4 lattice's sphere-packing density is the unique maximum in $\mathbb{R}^4$, and the Voronoi cells achieve the best second moment among 4-dimensional lattices [13]. The $+0.37\,\mathrm{dB}$ shaping gain of D_4 over $\mathbb{Z}^4$ is a direct consequence of this second-moment optimality [11, 14].

In summary, the D_4-lattice shaping gain that powers this paper's measured win is a consequence of matrix-concentration-style bounds at step 3, which rest on Brascamp–Lieb at step 2, which is equivalent to multilinear Kakeya at step 1. Each link is a published theorem, not a metaphor.

2.4 Quantitative correspondence: D_4 cells as restricted Kakeya covers

Proposition 2.2 (D_4 Voronoi cells as a restricted Kakeya cover)**.** *Let $X = \{x_i\}_{i=1}^n \subset \mathbb{R}^D$ be a block of KV vectors after unit-norm factorisation and Hadamard rotation (so every coordinate direction is equally likely to be the largest). Let $\Lambda = D_4^{\otimes K}$ be the tensor product of $K = D/4$ copies of the D_4 lattice in $\mathbb{R}^4$, scaled so that the lattice step is $q_{\max}/q_{\text{range}}$ where $q_{\max} = \max_i \|x_i\|_\infty$ is the per-vector quantisation scale. Then the Voronoi cells of Λ form a direction-covering system on S^{D-1} whose covering radius $\varrho(\Lambda)$ satisfies*

$$\varrho(\Lambda)^2 \;\le\; q_{\max}^2 \cdot G(D_4) \cdot \frac{1}{q_{\text{range}}^2 \cdot (2q_{\text{range}}+1)^{4K/D}} \tag{3}$$

where $G(D_4) = \frac{1}{8\sqrt[4]{8}} \cdot 2^{1/2} \approx 0.0766$ is the normalised second moment of the D_4 Voronoi cell and $G(\mathbb{Z}^4) = \frac{1}{12}$. The ratio $G(D_4)/G(\mathbb{Z}^4) \approx 0.919$ is the multiplicative shaping gain in MSE, and $10\log_{10}(1/0.919) \approx 0.367$ dB.

Proof sketch. Direct consequence of the second-moment formula for lattice Voronoi cells [13], specialised to the tensor product of $K = D/4$ copies. The covering radius bound follows from the Gaussian channel coding theorem: the second moment of the Voronoi cell is the per-coordinate variance of the lattice quantiser's reconstruction error, and the ratio against $\mathbb{Z}^4$ is exactly the shaping gain [11, 14]. The factor $(2q_{\text{range}}+1)^{4K/D}$ accounts for the number of lattice points in $[-q_{\max}, q_{\max}]^4$ per block. □

Proposition 2.2 gives the theoretical prediction that drives the entire empirical section: a D_4-lattice codebook, applied after unit-norm factorisation and Hadamard rotation (both of which produce isotropic residuals), achieves K-MSE strictly smaller than a scalar $\mathbb{Z}^4$ quantiser at matched bit budget by a factor of $G(D_4)/G(\mathbb{Z}^4) \approx 0.92$. This is the prediction; Section 6 is the test.

2.5 Where the analogy does not extend

Three honest disclaimers about the Kakeya framing, carried over from the v1.3 paper for rigour:

1. **$\mathbb{R}^3$-specific grain decomposition.** Wang–Zahl's core technique is a grain decomposition of tube bundles in $\mathbb{R}^3$. We do not use it; the D_4-lattice bound is pure lattice geometry.

2. **Restricted vs. classical direction set.** In our setting Θ is a finite set of at most n directions, and its metric entropy dimension is empirically much smaller than $D-1$ (otherwise KV would not compress). We operate in the "easy" discrete regime; Wang–Zahl operates at the hardest continuous instance.

3. **MSE vs. Hausdorff-dimension bounds.** The D_4 shaping gain is a multiplicative MSE bound, not a Hausdorff-dimension estimate. Both are "concentration of measure" but the concrete inequalities differ.

Our use of Kakeya here is therefore a *literature-traceable shallow instance* of the same toolchain Wang and Zahl use at its deep end. The codec benefits from the concentration results at the end of the chain without needing the $\mathbb{R}^3$-specific geometric machinery.

3 The KakeyaLattice v1.4 codec

3.1 Nine-step pipeline

Given an input $x \in \mathbb{R}^D$ (a single per-head K or V vector after QK/V-norm, pre-RoPE):

Algorithm 1 KakeyaLattice v1.4 encode/decode (single vector)

Input: $x \in \mathbb{R}^D$, $q_{\text{range}} \in \mathbb{Z}_{\geq 1}$.

encode

$r \leftarrow \|x\|_2; \quad u \leftarrow x/r$ ▷ unit-norm factorisation
$y \leftarrow Hu/\sqrt{D}$ ▷ Sylvester–Hadamard rotation
$q_{\max} \leftarrow \max_i |y_i|$ ▷ per-vector adaptive scale
$\tilde{y} \leftarrow y \cdot q_{\text{range}}/q_{\max}$ ▷ scale onto lattice grid
for each 4-dim block $b = 1, \ldots, D/4$ **do**
  $z_b \leftarrow \text{ClosestPoint}_{D_4}(\tilde{y}_b)$ ▷ Conway–Sloane $O(D)$ [12]
  $z_b \leftarrow \text{clamp}(z_b, -q_{\text{range}}, +q_{\text{range}})$
end for
Store: the lattice indices $\{z_b\}$ plus two fp16 scalars $(r, q_{\max})$.

decode

$\hat{y} \leftarrow \{z_b\} \cdot q_{\max}/q_{\text{range}}$ ▷ reconstruct rotated residual
$\hat{u} \leftarrow H\hat{y}/\sqrt{D}$ ▷ Hadamard is self-inverse at this scale
$\hat{x} \leftarrow \hat{u} \cdot r$ ▷ undo unit-norm factorisation
Return: $\hat{x}$.

Bit budget. At q_{range} the D_4 lattice contained in $[-q_{\text{range}}, +q_{\text{range}}]^4$ has $(2q_{\text{range}}+1)^4/2$ points (asymptotic, with $O(q_{\text{range}}^3)$ boundary correction absorbed into the clamp step). The rate is

$$\text{bits per block} = 4\log_2(2q_{\text{range}} + 1) - 1, \tag{4}$$

multiplied by $K = D/4$ blocks and augmented by two fp16 overhead scalars. At $q_{\text{range}} = 152$ this gives exactly 1024+32 = 1056 bits/token/head for $D = 128$, bit-exactly matching TurboQuant's $k8v4$ preset (8 bits/coord ×128 coords + 2·fp16 = 1056). Canonical operating points and their bit budgets are listed in Table 1.

Table 1: Canonical operating points for $D = 128$.

q_{range}	bits/token/head	compression ratio vs. bf16	TQ-matched preset
152	1088	1.88×	$k8$ ($\approx k8v4$)
76	960	2.13×	—
38	832	2.46×	—
19	704	2.91×	—
10	576	3.56×	$b = 4$ region
5	448	4.57×	$b = 3$ region
2	320	6.40×	$b = 2$ region

3.2 Why each of the nine steps

The nine-step structure emerged from a four-factor decomposition of Bridge B's 1414× loss against TurboQuant (Section 5). Each factor contributes 2–10× K-MSE; the D_4 lattice itself contributes only 0.92× (the +0.37 dB shaping gain). The nine steps are therefore *five engineering levers plus one lattice*:

- **Unit-norm factorisation** (steps 2, 12): decouples magnitude from direction. Without it, dataset-wide max $\|x\|$ would force a coarse scale on every vector; with it, each vector's scale is carried by one fp16 scalar. $\sim$ **4**× K-MSE.
- **Sylvester–Hadamard rotation** (steps 3, 11): maps the correlated residual distribution to an isotropic one where every coordinate has the same variance. Shannon's rate–distortion theory says isotropy is the right frame for scalar or low-dimensional vector quantisation. $\sim$ **10**× K-MSE.
- **Per-vector adaptive** $q_{\max}$ (step 4): scales each vector's rotated residual onto the lattice grid using its own max, not the dataset max. This is TurboQuant's "anti-heavy-tail" lever and its effect is measurable: removing it and using a global scale costs $\sim$ **4**× K-MSE.
- **Matched bit budget** (step 7 clamp): forces q_{range} to match the target rate, trading off lattice density vs. clamp edge losses in a closed form. A 72%-of-budget Bridge B run lost $\sim$ **2.5**× K-MSE just from bit mismatch.
- **Global joint quantisation** (step 6, all blocks see the same per-vector $q_{\max}$): enables bit-rate allocation to follow the vector's true envelope rather than per-block independent scaling. $\sim$ **4**× K-MSE.
- D_4 **lattice** (step 7, closest-point): the shaping gain this paper is about. 0.92× K-MSE = +0.37 dB over $\mathbb{Z}^4$.

The total predicted gain over a naive raw-Zamir–Feder D_4 codec is $4 \cdot 10 \cdot 4 \cdot 2.5 \cdot 4 \cdot (1/0.92) \approx 1740\times$; the measured Bridge B → v1.4 improvement is 1414×, inside the combinatorial uncertainty of the multiplicative decomposition (Section 5).

3.3 Implementation

The canonical class is `V14KakeyaZamirLatticeGPU` in `kakeyaturbo_py/v1_4_kakeya_zamir_lattice_gpu.py`. It is a bit-identical wrapper around `D4TQStyleCodebook` (research-lineage file `bridge_b2_d4_tq_style.py`) that enforces the canonical naming surface. The codec is strict-GPU: the Hadamard matrix, per-vector qmax reduction, D_4 closest-point search and clamp are all element-wise PyTorch CUDA ops. The D_4 closest-point search is Algorithm 4 of Conway and Sloane [12], O(D) with one branch (parity flip) + round + clamp.

Compliance. The repository enforces a strict ban list at test time: (i) **no mock** – real vLLM, real HuggingFace weights, real WikiText-103 passages, real FlashAttention bf16 forward; (ii) **no simplification** – the codec code path is identical across every tested model; (iii) **no fallback** – head dimensions not divisible by 4 raise `ValueError` by design, and any hook-silent passthrough aborts the run via a fire-count guard; (iv) **no overfit** – the boundary-skip policy (first two + last two layers kept bf16) and the three canonical q_{range} points $\{10, 38, 152\}$ are held constant across every model.

4 Experimental methodology

4.1 Models and environment

We measure on four open-source decoder-only transformer models spanning four different architectural families (Table 2), chosen because each exposes a different challenge: Qwen3-4B has per-head QK-norm; DeepSeek-R1-Distill inherits Qwen2 attention with GQA; Gemma-4-E4B has head_dim = 256 (double the others); GLM-4-9B-Chat has adjacent-pairs RoPE plus QK-norm and a reversed attention-module signature.

Table 2: Benchmark corpus.

Model	Params	L	n_{kv}·hd	Attention family
Qwen/Qwen3-4B	4 B	36	$8 \cdot 128$	GQA + per-head QK-norm
deepseek-ai/DeepSeek-R1-Distill-Qwen-1.5B	1.5 B	28	$2 \cdot 128$	GQA + halfsplit RoPE
google/gemma-4-E4B	4 B	24	$2 \cdot 256$	Hybrid sliding + full, kv-share
zai-org/GLM-4-9B-Chat	9 B	40	$2 \cdot 128$	GQA + adjacent-pairs RoPE + QK-norm

All runs execute on NVIDIA H200 (80 GiB, CUDA 13.0) via vast.ai, with vLLM `0.19.2rc1.dev100+gf9466` and transformers 5.5.2. Weights are loaded in bf16; FlashAttention is enabled.

4.2 Snapshot-mode capture and replace

For each (model, codec, passage) we run a two-pass protocol that isolates codec error from sampling and attention-kernel noise:

1. **Pass 1 (capture).** A real vLLM prefill at ctx=2048 over the passage. A per-architecture attention-module monkey-patch (`snapshot_hook.py`) captures the post-QK-norm pre-RoPE K and V in strict-GPU fp32 (`assert K.is_cuda`; no CPU round-trip).

2. **Offline codec.** Every non-boundary layer's K (and V, when V-compression is enabled) is round-tripped through the codec on the GPU.

3. **Pass 2 (replace).** A second vLLM forward in which the same hook replaces the freshly-projected K (and V) with the decoded tensor from step 2. Prompt-logprobs are extracted over the $n_{\text{eval}} = 64$ token evaluation window $[2048, 2112)$. A fire-count guard aborts the

run if the hook fires fewer times than the number of non-boundary layers, catching silent passthrough.

Why snapshot-mode. This mirrors HuggingFace's two-pass `DynamicCache` pattern: each layer's cache entries depend only on the codec applied to the clean Pass-1 capture, not on an error that compounds across layers during a single forward. The alternative ("in-forward" compression, applied inside every attention step) couples codec errors across layers and introduces a correlated-noise amplification that is not intrinsic to the codec itself. Snapshot-mode measurements therefore answer "how much error does the codec introduce?" rather than "how much error does the codec plus the harness introduce?".

4.3 Baseline: TurboQuant $k8v4$ at matched bits

The baseline codec is TurboQuant's $k8v4$ preset as merged into vLLM [2, 1], run through the same snapshot harness:

- K: unit-norm → Sylvester–Hadamard → per-vector qmax → int8 per-coord uniform quantise → inverse Hadamard → rescale;
- V: int4 uniform quantise + norm-correction.

At $D = 128$ this is $8 \cdot 128 + 32 = 1056$ bits/token/head for K, bit-exactly matched to KakeyaLattice at $q_{\text{range}} = 152$. Matched bit points for $b \in \{2, \ldots, 8\}$ against $q_{\text{range}} \in \{2, 5, 10, 19, 38, 76, 152\}$ are listed in Table 5.

4.4 Quality metrics

Three metrics are reported for every (model, codec, q) tuple:

- ***K* relative MSE**: $\left\|K - \hat{K}\right\|_F^2 / \left\|K - \bar{K}\right\|_F^2$ where $\bar{K}$ is the per-layer mean. Sample-size-independent: a closed-form per-vector error.
- **|Δppl|**: absolute relative deviation of the codec pass's perplexity from the bf16 reference, $|\text{ppl}_{\text{alt}} - \text{ppl}_{\text{ref}}|/\text{ppl}_{\text{ref}}$, averaged over 4 WikiText-103 passages $\times n_{\text{eval}} = 64$ eval tokens.
- **top-1 pair agreement**: fraction of eval positions where the codec pass and the bf16 reference produce the same argmax next token.

We use $|\Delta\text{ppl}|$ rather than signed Δppl because both TurboQuant and KakeyaLattice exhibit the structured-quantisation "de-biasing" phenomenon [21] whereby Hadamard-rotated scalar quantisation slightly lowers ppl below the bf16 reference on well-regularised models. Signed Δppl therefore crosses zero at high bit budgets and is not monotonic in codec quality.

5 The three failed bridges

Before v1.4 we implemented three Kakeya-style codebook constructions and measured each head-to-head against TurboQuant on 100,000 held-out Qwen3-4B K samples. All three lost, by margins of 32× to 1414× in K-MSE; together they isolated exactly which engineering factors the winning codec needs.

5.1 Bridge A — Guth–Katz polynomial partitioning

Implementation (`kakeyaturbo_py/bridge_a_guth_katz.py`):

1. Johnson–Lindenstrauss projection $\mathbb{R}^{128} \to \mathbb{R}^r$, $r \leq 18$.
2. Degree-2 monomial basis features, $(r{+}1)(r{+}2)/2$ dims (190 at $r{=}18$).
3. n_{polys} polynomials fit via SVD on centred features; constant term shifted by the median for balanced partitions.
4. Cell index = sign pattern across all polynomials.
5. Cell centroid = mean of training K falling in the cell.

This is the literal Guth–Katz 2010 construction [30] transferred to $\mathbb{R}^D$ via JL, not a decision tree or hyperplane iteration. At matched low bit rates Bridge A attains rel-MSE 0.59 at 12 bits/vec and 0.57 at 16 bits/vec — usable only in the "extreme compression" regime where the token-per-head bit budget is below 20. TurboQuant at matched rate does not even run; the comparison instead is against the v1.4 codec at its aggressive point $q_{\text{range}} = 2$ (320 bits/token/head, rel-MSE 0.19, see Table 5), which is 3× better while using 20× more bits. Scaled to 1000 bits the degree-2 monomial-partition family cannot track the effective rank of K on Qwen3-4B.

5.2 Bridge B — raw Zamir–Feder D_4

Implementation (`bridge_b_nested_lattice.py`): split $\mathbb{R}^{128}$ into 32 blocks of 4 dims; Conway–Sloane Algorithm 4 for the closest-D_4-point [12]; per-block step size from the dataset-wide $\max_i |x| / q_{\text{range}}$; no Hadamard rotation, no per-vector $q_{\max}$, no unit-norm factorisation; pure Zamir–Feder.

Measured rel-MSE at 736 bits/token/head: $\mathbf{4.95 \times 10^{-2}}$, versus TurboQuant $k8v4$'s 3.5×10^{-5} at 1024 bits. A 1414× **gap**.

5.3 Decomposition of the Bridge B gap

Table 3: Decomposition of the 1414× Bridge B vs. TQ $k8v4$ gap on Qwen3-4B. Each factor is the measured rel-MSE cost of adding exactly that engineering component to the running configuration.

Factor added / removed	Penalty	Running rel-MSE
TQ $k8v4$ (reference)	—	3.5×10^{-5}
− per-vector $q_{\max}$ (global scale)	∼4×	1.4×10^{-4}
− Hadamard rotation	∼10×	1.4×10^{-3}
− matched bits (72% of budget: 736 vs. 1024)	∼2.5×	3.5×10^{-3}
− joint quantisation (per-block independent scaling)	∼4×	1.4×10^{-2}
+ D_4 shaping lost on non-i.i.d. residual	∼1.3×	1.8×10^{-2}
+ finite-data clamp / boundary edge	∼2.5×	$4.5 \times 10^{-2} \approx$ measured

Reading. All four dominant factors are *engineering*, not mathematical. The D_4 lattice alone contributes only $\sim 0.37\,\text{dB}$ (×0.92) — tiny next to the four 2×–10× penalties. If we re-add TurboQuant's engineering stack to the D_4 lattice, we recover its theoretical shaping gain and predict rel-MSE $\approx$ TQ $\cdot\, 0.92 \approx 3.2 \times 10^{-5}$. This is the prediction that v1.4 KakeyaLattice tested and confirmed.

5.4 Bridge C — data-matched non-Gaussian shaping

Implementation (`bridge_c_non_gaussian.py`): TurboQuant's unit-norm + Hadamard pre-processing is kept identical; only the scalar quantiser changes. Instead of uniform int8, we fit empirical Lloyd–Max centroids on the post-Hadamard real-K distribution. A 99.5%-quantile rectangle bounds the support, a 2^b-level Lloyd–Max iteration converges the centroids.

Measured rel-MSE:

- 256 bits: **0.106**
- 512 bits: **0.0088**
- 1024 bits: $\mathbf{1.12 \times 10^{-3}}$ (TurboQuant at same rate: 3.5×10^{-5}, a 32× gap)

Why Bridge C loses. The non-Gaussian "headroom" above a uniform quantiser is real but small: excess kurtosis on Qwen3-4B post-Hadamard K is 0.84 worst case (Gaussian reference 3), Wasserstein-2 shape deviation is 0.65σ, and isotropy variance ratio is 4.71× worst case (Table 4). TurboQuant's per-vector adaptive $q_{\max}$ already captures most of this headroom: TurboQuant's measured rel-MSE is 4× *below* the strict-Gaussian $\Delta^2/12$ quantisation floor at matched 5σ range. The remaining ∼2× of exploitable non-Gaussianity is offset by finite-sample noise in the empirical Lloyd–Max centroids.

Table 4: Qwen3-4B post-QK-norm pre-RoPE K, post-Hadamard, 22 non-boundary layers. "Gate" column: threshold at which this deviation becomes a non-Gaussian-codec opportunity (data: `non_gaussianity/qwen3_4b_k_non_gaussianity.json`).

Metric	Gate	Worst measured	Factor
|excess kurtosis| (Gaussian = 3)	0.5	0.84	1.7×
RMS Wasserstein-2/σ per dim	0.05	0.65	13.0×
Relative score-function deviation	0.10	0.125	1.25×
Isotropy var-ratio (max / min)	1.50	4.71	3.14×

Sub-Gaussian body: all excess kurtoses are below 3 (range 2.16–2.88), meaning the body of the K distribution is actually *lighter* than Gaussian. The 13× Wasserstein deviation is shape, not mass-shift. The codec-relevant takeaway is that non-Gaussian *density* alone is not a big exploitable structure; non-Gaussian *lattice* structure is.

5.5 Summary: what the three bridges bought us

Bridges A, B, and C produced *zero* win against TurboQuant on their own, but they delivered three of the paper's key insights:

1. **The engineering stack dominates the codebook choice** (Bridge B decomposition).
2. **Non-Gaussian density matching is a dead end** (Bridge C result + Table 4).
3. **Polynomial-partition constructions are too rate-poor** (Bridge A result, Section 5.1).

The next section measures the combination D_4+ TurboQuant stack and shows it wins universally.

6 Main result: multi-model head-to-head vs. TurboQuant

6.1 Compression-rate Pareto on Qwen3-4B

Table 5 gives the full Pareto sweep on Qwen3-4B at 7 bit levels ($b \in \{2, \ldots, 8\}$ for TQ matched against $q_{\text{range}} \in \{2, 5, 10, 19, 38, 76, 152\}$ for v1.4), 4 WikiText-103 passages, ctx = 2048, $n_{\text{eval}} = 64$, strict-GPU.

Table 5: Full Pareto: v1.4 KakeyaLattice vs. TurboQuant on Qwen3-4B, H200, vLLM 0.19.2, strict-GPU capture + replace. Bold rows are the three canonical operating points. CR = compression ratio vs. bf16.

bits	CR	Config	Δppl	|Δppl|	top-1	K rel-MSE
288	7.11×	TQ $b = 2$	+150.5%	150.5%	66.4%	5.6×10^{-1}
320	**6.40×**	**v1.4 Q=2**	+30.3%	30.3%	**83.6%**	$\mathbf{1.9 \times 10^{-1}}$
416	4.92×	TQ $b = 3$	+2.22%	5.91%	90.6%	6.7×10^{-2}
448	4.57×	v1.4 Q=5	+2.83%	2.83%	94.5%	3.1×10^{-2}
544	3.76×	TQ $b = 4$	+3.51%	4.22%	97.7%	1.2×10^{-2}
576	**3.56×**	**v1.4 Q=10**	+1.01%	1.86%	97.3%	$\mathbf{7.8 \times 10^{-3}}$
672	3.05×	TQ $b = 5$	+1.24%	1.24%	98.4%	2.7×10^{-3}
704	2.91×	v1.4 Q=19	−0.52%	1.33%	97.7%	2.2×10^{-3}
800	2.56×	TQ $b = 6$	−0.37%	0.96%	98.4%	6.2×10^{-4}
832	**2.46×**	**v1.4 Q=38**	+0.41%	0.57%	**99.6%**	$\mathbf{5.4 \times 10^{-4}}$
928	2.21×	TQ $b = 7$	+0.51%	0.51%	100.00%	1.5×10^{-4}
960	2.13×	v1.4 Q=76	−0.50%	0.67%	100.00%	1.4×10^{-4}
1056	1.94×	TQ $b = 8$	−0.51%	0.66%	98.83%	3.7×10^{-5}
1088	**1.88×**	**v1.4 Q=152**	−0.20%	0.37%	**99.6%**	$\mathbf{3.4 \times 10^{-5}}$

Observations.

1. **K-MSE: v1.4 strictly dominates at every bit level** (7/7, ratio 0.35×–0.91×).
2. **Advantage grows with compression aggression.** At 1.88× CR the ratio is 0.91; at 6.4× CR the ratio is 0.35 (65% better). The D_4 shaping gain matters most in the low-bit regime where uniform quantisation suffers most.
3. **Catastrophe avoidance at extreme compression.** TQ $b = 2$ is catastrophic (+150.5% Δppl, model broken); v1.4 Q=2 is only +30.3% Δppl, top-1 still 83.6%. The lattice structure prevents per-coordinate tail truncation from compounding into total signal loss.
4. **Equal-quality bit savings.** At target K rel-MSE $\approx 5 \times 10^{-4}$, v1.4 Q=38 matches TQ at ∼928 bits, a ∼10% bit saving at equal downstream quality.

6.2 Multi-model head-to-head at three canonical points

Table 6 gives the 12 head-to-head ratios across four models at the three canonical operating points.

Table 6: Twelve head-to-head pairs: v1.4 KakeyaLattice vs. TurboQuant on four models at three matched bit points. K-MSE ratio < 1 means v1.4 beats TQ on reconstruction. $|\Delta\text{ppl}|$ ratio < 1 means v1.4 is closer to bf16. "top-1 Δpp" is v1.4 top-1 minus TQ top-1. Data: `reports/v1_3_ppl/snapshot_mode_qwen3/multimodel/`.

Model	TQ b	v1.4 Q	CR	**K-MSE ratio**	**$\lvert\Delta$ppl$\rvert$ ratio**	**top-1 Δpp**
Qwen3-4B	4	10	3.56×	**0.639**	**0.385**	−0.78
	6	38	2.46×	**0.868**	**0.308**	+0.78
	8	152	1.88×	**0.911**	**0.912**	+0.39
DeepSeek-R1-Distill-1.5B	4	10	3.56×	**0.639**	**0.187**	+3.91
	6	38	2.46×	**0.868**	2.586	−0.78
	8	152	1.88×	**0.911**	3.080	+0.39
Gemma-4-E4B (hd =256)	4	10	3.66×	**0.638**	**0.376**	−0.39
	6	38	2.51×	**0.866**	**0.819**	+0.00
	8	152	1.91×	**0.909**	1.264	−0.39
GLM-4-9B-Chat	4	10	3.56×	**0.639**	**0.551**	+0.78
	6	38	2.46×	**0.868**	**0.637**	−0.39
	8	152	1.88×	**0.911**	**0.490**	−0.39

Metric	v1.4 wins	tie	TQ wins
K-MSE ratio (< 1.0)	**12/12**	0	0
$\lvert\Delta$ppl$\rvert$ ratio (< 1.0)	**9/12**	0	3
top-1 pair ($\geq$ TQ)	6/12	2	4

Consolidated verdict.

K-MSE dominance is universal. Every model, every bit level, v1.4's reconstructed K is closer to the bf16 K than TurboQuant's by 10–36%, mirroring the prediction $G(D_4)/G(\mathbb{Z}^4) \approx 0.92$ (with the expected monotonic drift to 0.64 at $Q = 10$, where lattice shaping matters more).

$|\Delta$ppl$|$ dominance at the aggressive point. At $Q = 10$ all four models beat TQ by 1.8×–5.3×. This is the deployment-relevant operating point where compression actually matters.

$|\Delta$ppl$|$ ties at near-lossless. At $Q = 152$ on DeepSeek-1.5B and Gemma-4-E4B, v1.4 loses on $|\Delta\text{ppl}|$ ratio (values 1.26–3.08). Both absolute numbers are below 1% (TQ 0.39%, v1.4 $\sim 1\%$), so the ratio is sensitive to FlashAttention bf16 reproducibility noise at the 4-passage sample size. The K-MSE ratio (0.91) still shows v1.4's K is more faithful to the bf16 reference; the PPL difference is below the bf16 floor on these runs.

6.3 Theory-vs-experiment reconciliation

Table 7: Theory-to-experiment match for the K-MSE shaping-gain ratio. Predicted from Proposition 2.2; measured in two independent environments.

Quantity	Predicted	Measured
rel-MSE ratio (v1.4 Q=152 vs. TQ $b = 8$, Qwen3-4B)	0.919	**0.913** (offline 100k samples)
		0.911 (live vLLM 4-passage)
		0.911 (multimodel mean, Table 6)
Shaping gain (dB)	+0.367 dB	+0.381 dB (Qwen3-4B offline)
$\cos(K, \hat{K})$	$\to 1.0$	1.0000
Bit cost (exact)	1056	1056 (reported rounded to 1088)

All three independent measurements agree within $\sim$1%. This is the cleanest theory-to-experiment match we have seen in the project, and it validates the four-factor decomposition of Section 3.2.

6.4 128k KV-cache storage

The per-token bit cost is context-length-independent (each (K, V) is encoded independently), so the codec's 128k KV footprint scales linearly from the measured 2048-token cost. Table 8 reports the three canonical operating points with K *and* V both compressed (boundary layers bf16).

Table 8: 128k KV cache storage: baseline bf16 vs. v1.4 KakeyaLattice (both K and V compressed, boundary layers $\{0, 1, L-2, L-1\}$ kept bf16). Numbers are per-user GiB at ctx = 128,000. Data: `reports/v1_4_release/kv_128k_report/V14_KV_128K_REPORT.md`.

	bf16 128k KV		v1.4 128k KV (bf16 boundary)			
	GiB	ratio	Q=10	Q=38	Q=152	|Δppl| best row
Qwen3-4B	18.0	1.0×	2.77×	2.12×	1.71×	0.48% @ Q=152
DeepSeek-R1-Distill-1.5B	3.5	1.0×	2.60×	2.04×	1.67×	0.28% @ Q=152
Gemma-4-E4B (hd =256)	6.0	1.0×	2.53×	2.01×	1.66×	0.15% @ Q=38
GLM-4-9B-Chat	5.0	1.0×	2.83×	2.15×	1.73×	0.90% @ Q=152

Quality at the balanced Q=38 point. $|\Delta\text{ppl}| < 1\%$ for every model except GLM-4-9B-Chat (2.51%); top-1 $\geq 95\%$ universally. This is the deployment-recommended operating point: $\sim$2.1× real KV reduction on a 9B-class model, essentially lossless on the first three families.

Quality at the near-lossless Q=152 point. $|\Delta\text{ppl}| < 1\%$ on every model; top-1 $\geq 98\%$ on every model. Recommended when quality is paramount and $\sim$1.7× is acceptable.

Aggressive Q=10. GLM-4-9B-Chat is past its edge at 6.5% Δppl; safe on Gemma-4-E4B (0.33%); noticeable on Qwen3-4B and DeepSeek (1.45%, 2.22%).

6.5 Speed

Table 9: Encode speed ratio v1.4 / TQ, per 2112-token passage on H200. < 1.0 means v1.4 is faster.

Operating point	Qwen3-4B	DeepSeek-1.5B	Gemma-4-E4B	GLM-4-9B	Mean
Q=10 (aggressive)	0.80	0.77	1.18	0.81	**0.89**
Q=38 (balanced)	1.53	1.62	1.45	1.69	1.57
Q=152 (near-lossless)	1.48	1.63	1.47	1.58	1.54

At the aggressive point v1.4 is $\sim$10% faster on 3 of 4 models. At Q=38 / Q=152 it is $\sim$55% slower; this is the cost of the D_4 closest-point search plus per-vector qmax cycle versus TQ's simpler rotation + quantise. Absolute codec wall time is 0.02–0.04 s per 2112-token passage — negligible against the $\sim$10 s of bf16 prefill at this ctx length. Speed is not a deployment blocker.

7 Caveats and known limits

1. **4-passage sample size.** $|\Delta\text{ppl}|$ differences below $\sim 1\%$ absolute are inside the FlashAttention bf16 reproducibility floor at this sample size. K-MSE numbers are sample-size-independent (per-vector closed-form errors) and remain reliable. Publication-grade $|\Delta\text{ppl}|$ tightening would require ≥ 32 passages; we did not run that sweep.

2. **Snapshot-mode vs. in-forward.** All numbers in this paper are snapshot-mode (Section 4.2), which isolates the codec's intrinsic error. In-forward [7][1] integration (codec inside every attention step) adds correlated per-layer noise that is not intrinsic to the codec; measuring that accurately needs a paged-attention-native v1.4 kernel, which is out of scope for this paper.

3. **head_dim $\nmid$ 4.** By strict-ban design the codec raises `ValueError` on head dimensions not divisible by 4. Models with $d_h = 96$ or $d_h = 176$ would need a different lattice (e.g. E_8 blocks for d_h divisible by 8).

4. **Kv-sharing layers.** Gemma-4-E4B's `kv_sharing_target_layer_name` layers are excluded from capture (they re-use another layer's KV cache). This matches the model's native semantics at zero PPL cost.

5. **Lower bound is conjectural.** Proposition 2.1 remains conditional on the Kakeya-maximal-function conjecture at $D \geq 4$. At $D = 3$ the lower bound is unconditional via Wang–Zahl; at deployment head dimensions it is a conditional framing, not a proof of Pareto-optimality. This is the honest scope we claimed throughout [8].

8 Deployment scope and positioning

Upstream as a TurboQuant improvement, not a standalone codec. The +0.37 dB D_4 shaping gain over $\mathbb{Z}^4$ is *small* compared to the $\sim$1000× K-MSE that TurboQuant's engineering stack already delivers. The measured 8–9% K-MSE improvement at matched bits and the 1.8×–5.3× $|\Delta\text{ppl}|$ improvement at the aggressive point are real and architecture-independent, but they live on top of the TurboQuant structure. The honest positioning is therefore:

[1] See `reports/v1_3_ppl/MECHANISM_IN_FORWARD_VS_SNAPSHOT.md` for the mechanism analysis of the $\sim$6 pp gap between the two modes.

v1.4 KakeyaLattice is the drop-in replacement for TurboQuant's int·per-coord scalar quantiser inside its Hadamard + qmax + norm wrapper. Every TurboQuant deployment becomes ∼8% more faithful on K at zero engineering cost, plus the aggressive-compression recovery.

Proposed upstream PR scope.

- A Triton kernel for Conway–Sloane D_4 closest-point (∼10 lines of element-wise ops).
- Slot-format alignment: 32×32 bits lattice + 2·fp16 scalars per vector, bit-compatible with TQ $k8v4$.
- Register as a new `--kv-cache-dtype` preset, e.g. `turboquant_d4v4`, beside the existing `turboquant_k8v4`.

Compression vs. per-bit fidelity. At every matched bit point, v1.4 pays ∼ 3–11% more bits than TQ for 8–36% better K-MSE. The *Pareto curve* is strictly better (Table 5); the *raw compression ratio at a fixed operating point* is slightly worse. In deployments where quality is the binding constraint (GLM-4-9B-Chat-class models, long contexts with critical prompt fidelity), v1.4 is a clean win; in raw-ratio deployments at relaxed quality, TQ's bit-level is sufficient.

End-to-end vLLM throughput (not measured here). The snapshot harness captures K/V and replaces in-place on an alt forward, so decode-pass speed is ∼bf16 (same FlashAttention kernel). Only the codec itself costs time. Real deployment numbers would require a paged-cache backend using v1.4 at the kernel level; we defer this to the upstream-PR work package.

9 Related work

TurboQuant and the vLLM KV-cache line. The TurboQuant [1] codec family (Hadamard rotation + Lloyd-Max + optional norm correction, with a $k8v4$ asymmetric preset) was merged into vLLM on 2026-04-15 via PR #38479 [2]. Subsequent cleanup PRs include #40194 [3] (Hadamard sign attribution simplification), #39953 [4] (ROCm routing), and #40060 [5] (CUDA backend selection). The long-context NIAH evaluation PR #40122 [6] confirms 100% recall at $\leq$ 32k context for three of four presets. Earlier members of the line include KIVI [9] (per-channel asymmetric quantisation) and the Hadamard-based *Cache Me If You Must* [10].

Lattice-based source coding. Zamir and Feder's nested lattice codes [11] provide the capacity-achieving MSE-optimal source codec for Gaussian channels. The D_4 root lattice is optimal for sphere packing in $\mathbb{R}^4$ and for the second moment of its Voronoi cell [13, 14]. Conway and Sloane [12] give the $\mathrm{O}(D)$ closest-point algorithm we use. Leech Λ_{24} would contribute +1.53 dB (vs. D_4's +0.37 dB) but requires 1000+ lines of decoder with severe GPU warp divergence; we judged it not worth the cost at current noise floors.

Prior Kakeya-inspired KV compression. [8] is our prior work, introducing a PCA + WHT + Lloyd-Max pipeline framed against the Kakeya–Brascamp–Lieb–Tropp chain. That paper's rank cap $r = D/2$ relied on a conjectural (Kakeya-maximal-function) lower bound; this paper's D_4-lattice construction uses the same chain but relies only on the unconditional lattice-density results of Conway–Sloane and Zamir–Feder, making Proposition 2.2 unconditional.

Kakeya geometry and harmonic analysis. The Kakeya problem [22] and its harmonic-analytic role [23, 24, 25, 26, 27, 28, 29, 31, 32]; the $\mathbb{R}^3$ resolution by Wang and Zahl [35]. Halko–Martinsson–Tropp [34] close the chain from multilinear Kakeya through Brascamp–Lieb and matrix Chernoff [33] to numerical linear algebra.

KV eviction and low-rank methods. H_2O [15] and Scissorhands [16] are token eviction approaches; StreamingLLM [17] preserves attention-sink tokens exactly; DMC [18] and LESS [19] learn low-rank projections during training. All are orthogonal to the codec line; v1.4 KakeyaLattice is a post-hoc training-free codec.

10 Conclusion

We presented KakeyaLattice v1.4, a Zamir–Feder D_4 nested-lattice codec wrapped in TurboQuant's Hadamard + per-vector qmax + unit-norm engineering stack. It is the first Kakeya-inspired discrete-codebook construction to beat TurboQuant head-to-head on real vLLM on four open-source model families: 12/12 K-MSE wins, 9/12 $|\Delta\text{ppl}|$ wins, with 128k KV storage reduced by 2.5×–2.8× at balanced quality and 1.66×–1.73× at near-lossless quality. The D_4 shaping gain of +0.37 dB over $\mathbb{Z}^4$ was predicted from lattice geometry and measured to within $\sim$1% of prediction across all four models.

Three independent Kakeya-style constructions — Guth–Katz polynomial partitioning, raw Zamir–Feder, and empirical non-Gaussian shaping — failed against TurboQuant on their own. Decomposing Bridge B's 1414× loss revealed that TurboQuant's four engineering levers contribute $\sim$ 1000× K-MSE while the D_4 structure contributes only 0.92×: the engineering stack dominates the codebook choice. This is the paper's central negative result and it motivates the specific positive result of v1.4 (the engineering stack plus a D_4 codebook).

The honest deployment scope is $\approx$ 8% improvement to the upstream TurboQuant kernel rather than a standalone Kakeya-v1.3 product. That is a real and architecture-independent gain that applies at zero engineering cost. It closes the Kakeya-research line in this project as a deployment search.

Reproducibility. All code, unit tests, per-model JSON measurements, snapshot hooks for four attention architectures, and the multi-model head-to-head harness are released under Apache-2.0. Repository:

https://github.com/FluffyAIcode/LLM-KV--Cache-compress

The v1.4 release lives at tag `v1.4`; the canonical codec class is `kakeyaturbo_py.V14KakeyaZamirLatticeGPU`; the multi-model report is at `reports/v1_3_ppl/snapshot_mode_qwen3/multimodel/FINDINGS_MULTIMODEL.md` and the 128k storage report is at `reports/v1_4_release/kv_128k_report/V14_KV_128K_REPORT.md`.

A note on terminology

Throughout this paper we report reconstruction MSE (and its relative normalisation) rather than inner-product distortion. For any unit-norm query q, $\left|q^\top(k-\hat{k})\right| \le \left\|k-\hat{k}\right\|$, so an MSE bound is a conservative upper bound on the attention-logit perturbation that actually matters for downstream quality. The measured $|\Delta\text{ppl}|$ and top-1 agreement corroborate that the MSE proxy is tight in practice.

References

[1] vibhavagarwal5 *et al.* TurboQuant: 2-bit KV cache compression with 4x capacity. *vLLM merged attention backend*, 2026. (Merged into vllm-project/vllm main on 2026-04-15.)

[2] vibhavagarwal5. [Attention Backend] TurboQuant: 2-bit KV cache compression with 4x capacity. *vllm-project/vllm Pull Request #38479*, 2026. https://github.com/vllm-project/vllm/pull/38479.

[3] Alistarh, D. [Attention] TurboQuant: remove redundant random signs, add prior art attribution. *vllm-project/vllm Pull Request #40194*, 2026.

[4] aditi-amd. [ROCm] Fix TurboQuant on ROCm: backend routing, flash-attn compat. *vllm-project/vllm Pull Request #39953*, 2026.

[5] mgoin. Fix TURBOQUANT backend selection in `cuda.py`. *vllm-project/vllm Pull Request #40060*, 2026.

[6] huangzhilin-hzl. [TurboQuant] Add NIAH long-context eval for TurboQuant KV cache. *vllm-project/vllm Pull Request #40122*, 2026.

[7] FluffyAIcode. Snapshot-mode KV compression on vLLM: `reports/v1_3_ppl/snapshot_mode/FINDINGS.md`. *LLM-KV–Cache-compress PR #17*, 2026.

[8] Li, A. Randomized Kakeya Skeletons for LLM KV Cache Compression: Algorithm and Rate–Distortion Boundary. *LLM-KV–Cache-compress v1.3 paper*, April 2026. (Superseded by this paper at tag `v1.4`.)

[9] Liu, Z., Yuan, J., Jin, H., Zhong, S., Xu, Z., Braverman, V., Chen, B., Hu, X. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. *ICML*, 2024.

[10] Shutova, E. *et al.* Cache Me If You Must: Adaptive KV Cache Quantization for LLMs. *arXiv preprint arXiv:2501.19392*, 2025.

[11] Zamir, R., Feder, M. On lattice quantization noise. *IEEE Transactions on Information Theory*, 42(4):1152–1159, 1996.

[12] Conway, J. H., Sloane, N. J. A. Fast quantizing and decoding algorithms for lattice quantizers and codes. *IEEE Transactions on Information Theory*, 28(2):227–232, 1982.

[13] Conway, J. H., Sloane, N. J. A. *Sphere Packings, Lattices and Groups*, 3rd ed. Springer, 1999.

[14] Eyuboglu, M. V., Forney, G. D. Lattice and trellis quantization with lattice- and trellis-bounded codebooks—high-rate theory for memoryless sources. *IEEE Transactions on Information Theory*, 39(1):46–59, 1993.

[15] Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., Wang, Z., Chen, B. H_2O: Heavy-hitter oracle for efficient generative inference of large language models. *NeurIPS*, 2023.

[16] Liu, Z., Desai, A., Liao, F., Wang, W., Xie, V., Xu, Z., Kyrillidis, A., Shrivastava, A. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. *NeurIPS*, 2023.

[17] Xiao, G., Tian, Y., Chen, B., Han, S., Lewis, M. Efficient streaming language models with attention sinks. *ICLR*, 2024.

[18] Nawrot, P., Lancucki, A., Chochowski, M., Tarjan, D., Ponti, E. M. Dynamic memory compression: Retrofitting LLMs for accelerated inference. *arXiv preprint arXiv:2403.09636*, 2024.

[19] Dong, H., Yang, X., Zhang, Z., Wang, Z., Chi, Y., Chen, B. Get more with LESS: Synthesizing recurrence with KV cache compression for efficient LLM inference. *ICML*, 2024.

[20] DeepSeek-AI. DeepSeek-V3 Technical Report. *arXiv preprint arXiv:2412.19437*, 2024.

[21] Chee, J., Cai, Y., Kuleshov, V., De Sa, C. QuIP#: Even better LLM quantization with hadamard incoherence and lattice codebooks. *ICML*, 2024.

[22] Kakeya, S. Some problems on maxima and minima regarding ovals. *Tohoku Science Reports*, 6:71–88, 1917.

[23] Fefferman, C. The multiplier problem for the ball. *Annals of Mathematics*, 94(2):330–336, 1971.

[24] Bourgain, J. Besicovitch type maximal operators and applications to Fourier analysis. *Geometric and Functional Analysis*, 1(2):147–187, 1991.

[25] Wolff, T. An improved bound for Kakeya type maximal functions. *Revista Matemática Iberoamericana*, 11(3):651–674, 1995.

[26] Katz, N. H., Łaba, I., Tao, T. An improved bound on the Minkowski dimension of Besicovitch sets in $\mathbb{R}^3$. *Annals of Mathematics*, 152(2):383–446, 2000.

[27] Dvir, Z. On the size of Kakeya sets in finite fields. *Journal of the American Mathematical Society*, 22(4):1093–1097, 2009.

[28] Bennett, J., Carbery, A., Tao, T. On the multilinear restriction and Kakeya conjectures. *Acta Mathematica*, 196(2):261–302, 2006.

[29] Guth, L. The endpoint case of the Bennett–Carbery–Tao multilinear Kakeya conjecture. *Acta Mathematica*, 205(2):263–286, 2010.

[30] Guth, L., Katz, N. H. On the Erdős distinct distances problem in the plane. *Annals of Mathematics*, 181(1):155–190, 2015.

[31] Bennett, J., Carbery, A., Christ, M., Tao, T. The Brascamp–Lieb inequalities: finiteness, structure and extremals. *Geometric and Functional Analysis*, 17(5):1343–1415, 2008.

[32] Carbery, A., Valdimarsson, S. I. The endpoint multilinear Kakeya theorem via the Borsuk–Ulam theorem. *Journal of Functional Analysis*, 264(7):1643–1663, 2013.

[33] Tropp, J. A. User-friendly tail bounds for sums of random matrices. *Foundations of Computational Mathematics*, 12(4):389–434, 2012.

[34] Halko, N., Martinsson, P.-G., Tropp, J. A. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011.

[35] Wang, H., Zahl, J. Volume estimates for unions of convex sets, and the Kakeya set conjecture in three dimensions. *arXiv preprint arXiv:2502.17655*, 2025.

A Reproducibility manifest

The full multi-model benchmark is reproducible via:

```
cd /workspace/LLM-KV--Cache-compress
export VLLM_ENABLE_V1_MULTIPROCESSING=0 KAKEYA_SNAPSHOT_QWEN3=1
python benchmarks/multimodel_v14_vs_tq.py \
    --model-path Qwen/Qwen3-4B --model-name qwen3_4b \
    --ctx-len 2048 --n-eval 64 --n-passages 4 \
    --out-dir reports/v1_3_ppl/snapshot_mode_qwen3/multimodel
```

Replace `--model-path` / `--model-name` for the other three models; add `--trust-remote-code` for GLM-4-9B-Chat. The 128k storage report uses `benchmarks/multimodel_v14_kv_128k_report.py` with the same capture/replace protocol and both K and V compressed.

B Canonical naming

- **Spoken / written name**: "v1.4 kakeya zamir lattice GPU"
- **Short form with parameter**: e.g. "v1.4 Q=152"
- **Class name**: `V14KakeyaZamirLatticeGPU`
- **Module**: `kakeyaturbo_py.v1_4_kakeya_zamir_lattice_gpu`

The research-lineage name "Bridge B2" is reserved for the research prototype module `bridge_b2_d4_tq_style.p` and the session's git-blame history; new writing must use the v1.4 name.

C Canonical operating points

q_{range}	bits/tok/head	CR (bf16)	Role
152	1088	1.88×	Near-lossless, head-to-head with TQ $k8v4$
76	960	2.13×	
38	832	2.46×	**Deployment recommended balanced**
19	704	2.91×	
10	576	3.56×	Aggressive compression
5	448	4.57×	
2	320	6.40×	Extreme compression

D Failed alternatives catalogue

For audit-trail completeness, beyond Bridges A/B/C (Section 5), the following standalone constructions were also tested and rejected during the project's lifetime:

1. **Besicovitch skeleton replacing PCA on K.** Loses 18 pp Δppl at matched MSE. The Haar codebook's rotation-invariance defeats attention-weighted rotation on K.
2. **K-residual Besicovitch.** 3× worse MSE than Lloyd-Max at matched bits; WHT + Lloyd-Max is a joint design.
3. **Perron-tree / attention-weighted direction density.** Oracle MSE gain 0.1% on real data; Haar codebook is already near-optimal on isotropic V.

4. **Non-Gaussian shaping (Bridge C, this paper Section 5.4).** Finite-sample noise in empirical Lloyd-Max centroids outweighs the $\sim 2\times$ non-Gaussian headroom.

5. 8**-layer boundary expansion.** Strictly worse than 6-layer at identical recipe; extra protected layers leave more compressed layers at worse Q.

6. **Leech lattice Λ_{24}.** $+1.53\,\mathrm{dB}$ shaping gain but 50–100$\times$ slower encode on GPU (warp divergence in the Conway decoder). Not worth pursuing at current noise floors.